DATA ENGINEERING

dbt
Visual Guide

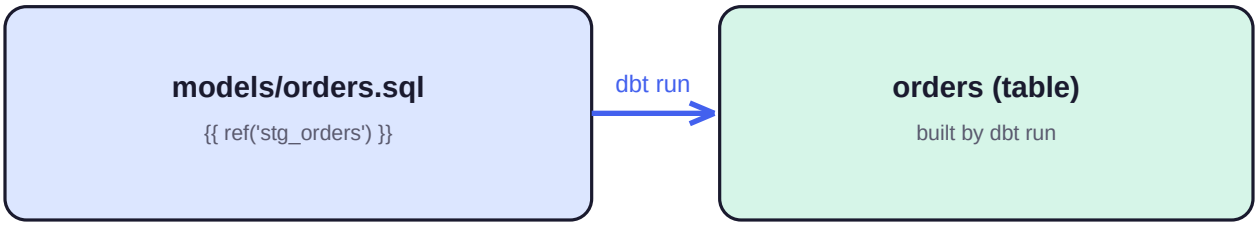
Analytics engineering: SQL models,
tests, and docs — with software discipline.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Models: SQL + Jinja

Each model is a SELECT that builds one table or view.



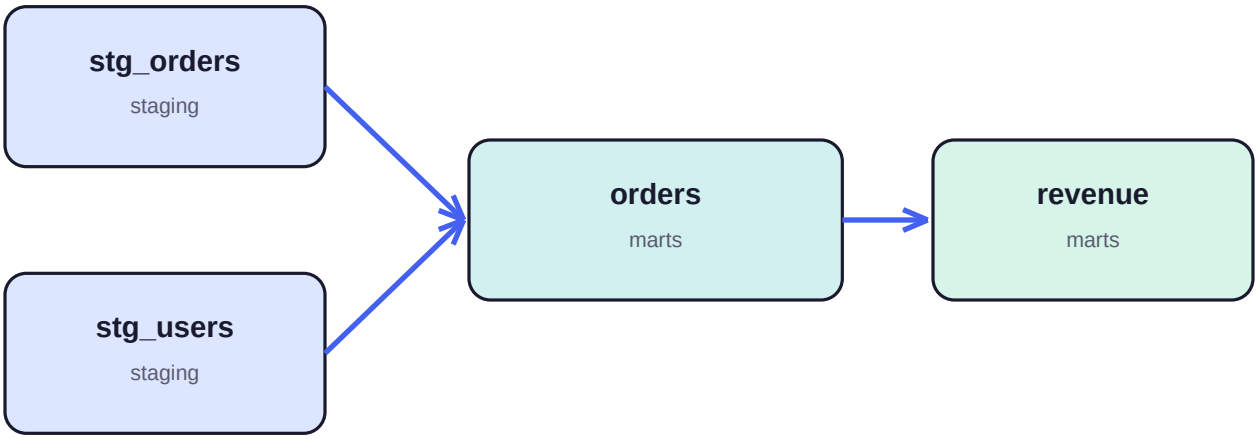

```
SELECT id, total FROM {{ ref('stg_orders') }}
WHERE total > 0
```

ref() wires dependencies automatically — dbt builds the DAG from your SQL. Jinja adds loops, macros, and environment-aware logic.

STEP 2/6

The Model DAG

dbt resolves refs into an execution graph and runs it in dependency order.



Independent models run in parallel; dependents wait. dbt run --select orders+ builds a model and everything downstream of it.

- Staging: 1:1 with sources, light cleaning only
- Intermediate: reusable business logic
- Marts: one model per business entity, the BI layer

STEP 3/6

Materializations

How each model is persisted — pick per model.

view

Always fresh, zero storage. Slow for heavy logic.

table

Rebuilt fully each run. Simple, expensive at scale.

incremental

Only new/changed rows appended. The workhorse.

ephemeral

CTE in downstream models. Never persisted.

Incremental models need an is_incremental() filter and a unique key — get those wrong and you silently duplicate rows.

STEP 4/6

Tests: Contracts on Your Data

Generic and custom tests run after every build.

unique
No duplicate keys

not_null
No missing values

accepted_values
Status in a known set

relationships
FK actually exists

```
models:■  - name: orders■    tests:■      - unique:■          column_name: id
```

Tests are queries that must return zero rows. dbt test fails the run on violations — bad data never reaches the marts silently.

STEP 5/6

Docs Generation

Documented models become a searchable data catalog for free.

dbt docs generate

static site from model + column descriptions

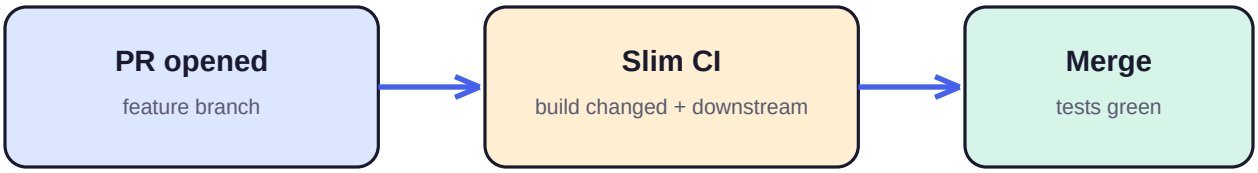
```
description: "One row per completed order"
columns:■  - name: total■    description: 'USD, tax included'
```

Descriptions live next to the SQL in YAML — the docs site renders the DAG, column lineage, and tests together. Documentation that can't drift.

STEP 6/6

CI for Analytics Code

Pull requests build and test changed models on a schedule-safe clone.



Slim CI (state:modified+) builds only what the PR touches plus downstream — fast feedback without rebuilding the warehouse.

- dbt build = run + test + snapshot + seed in dependency order
- Snapshots capture slowly-changing dimensions over time
- Seeds version-control small CSVs (mappings, lookups)

STEP ✓I✓

Key Takeaways

dbt — Remember

- Models are SELECT statements; ref() builds the dependency DAG automatically.
- Layer models: staging (raw 1:1) → intermediate (logic) → marts (business entities).
- Choose materializations deliberately: views for freshness, incremental for scale.
- Tests are zero-row queries that fail the build — data contracts enforced in CI.
- Docs generate from YAML next to the SQL; slim CI tests only what changed.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev